Kernel: SageMath 10.9

In [1]:

```
from itertools import combinations
# Sign of a permutation
def signs(perm):
    s = 1
    l = len(perm)
    for i in range(l):
        for j in range(i+1,l):
            if perm[i] > perm[j]:
                s *= -1
    return s

# Wedge Product
    prod = {}
    for i,j in combinations(range(4),2):
        value = u[i]*v[j] - u[j]*v[i]
        if value != 0:
            prod[(i,j)] = value
    return prod
# Coefficient Function
def coef(w,i,j):
    indices = sorted([i,j])
    if len(set([i,j])) < 2:
        return 0
    else:
        return signs([i,j]) * w.get(tuple(indices),0)
# Plucker Relation
def plucker(w):
    viol = []
    val = (coef(w,0,1)*coef(w,2,3)
          -coef(w,0,2)*coef(w,1,3)
          +coef(w,0,3)*coef(w,1,2))
    if val != 0:
        viol.append({"Value":val})
    return viol
# Decomposability Test
def is_decomposable(w):
    if len(plucker(w)) == 0:
        return True
    else:
        return False


def decompose(w, verbose=True):
    if not is_decomposable(w):
        if verbose:
            print("Plucker relation NOT satisfied -> omega is non-
decomposable.")
            print("  value =", plucker(w))
        return None

    a12 = coef(w,0,1)
    a13 = coef(w,0,2)
    a14 = coef(w,0,3)
    a23 = coef(w,1,2)
```

```
    a24 = coef(w,1,3)
    a34 = coef(w,2,3)

    ############################################################
    # Case 1: a12 = a13 = a23 = 0
    # omega = (a14 e0 + a24 e1 + a34 e2) ^ e3
    ############################################################
    if a12 == 0 and a13 == 0 and a23 == 0:
        case = "Case 1: a12=a13=a23=0"
        u = (a14, a24, a34, 0)
        v = (0, 0, 0, 1)

    ############################################################
    # Case 2: a14 != 0, a12 != 0
    # omega = (e0 + a24/a14 e1 + a34/a14 e2) ^ (a14 e3 + a12 e1 + a13 e2)
    ############################################################
    elif a14 != 0 and a12 != 0:
        case = "Case 2: a14!=0, a12!=0"
        u = (1, a24/a14, a34/a14, 0)
        v = (0, a12, a13, a14)

    ############################################################
    # Case 3: a14 != 0, a12 = 0, a13 != 0
    # same formula as Case 2 (see note above)
    ############################################################
    elif a14 != 0 and a12 == 0 and a13 != 0:
        case = "Case 3: a14!=0, a12=0, a13!=0"
        u = (1, a24/a14, a34/a14, 0)
        v = (0, a12, a13, a14)

    ############################################################
    # Case 4: a14 != 0, a12 = 0, a13 = 0, a23 != 0
    # UNREACHABLE if the Plucker relation holds (see note above)
    ############################################################
#     elif a14 != 0 and a12 == 0 and a13 == 0 and a23 != 0:
#         case = "Case 4: a14!=0, a12=0, a13=0, a23!=0  [should not occur]"
#         print("WARNING: reached Case 4 -- this contradicts the Plucker "
#               "relation (a14*a23 != 0 with a12=a13=0). Check input.")
#         u = (1, a24/a14, a34/a14, 0)
#         v = (0, a12, a13, a14)

    ############################################################
    # Case 5: a14 = 0, a12 != 0
    # omega = (a12 e1 + a13 e2) ^ (a24/a12 e3 + a23/a12 e2 - e0)
    ############################################################
    elif a14 == 0 and a12 != 0:
        case = "Case 5: a14=0, a12!=0"
        u = (0, a12, a13, 0)
        v = (-1, 0, a23/a12, a24/a12)

    ############################################################
    # Case 6: a14 = 0, a12 = 0, a13 != 0
    # omega = (a13 e2 + a12 e1) ^ (a34/a13 e3 - a23/a13 e1 - e0)
    ############################################################
    elif a14 == 0 and a12 == 0 and a13 != 0:
        case = "Case 6: a14=0, a12=0, a13!=0"
        u = (0, a12, a13, 0)
```

```
        v = (-1, -a23/a13, 0, a34/a13)

    ############################################################
    # Case 7: a14 = 0, a12 = 0, a13 = 0, a23 != 0
    # CORRECTED formula: original report drops the a24 term and
    # is only valid when a24 = 0, which is not forced here (unlike
    # Case 6, where a24 = 0 is forced by the Plucker relation).
    # omega = (e1 - a34/a23 e3) ^ (a23 e2 + a24 e3)
    ############################################################
    elif a14 == 0 and a12 == 0 and a13 == 0 and a23 != 0:
        case = "Case 7: a14=0, a12=0, a13=0, a23!=0  [corrected]"
        u = (0, 1, 0, -a34/a23)
        v = (0, 0, a23, a24)

    else:
        case = "omega = 0"
        u = (0,0,0,0)
        v = (0,0,0,0)

    if verbose:
        print(case)
        print("omega = u ^ v")
        print("  u =", u)
        print("  v =", v)

    return u, v

####################################################################
# Verification: reconstruct omega from (u,v) and compare
####################################################################
def verify_decompose(w):
    result = decompose(w)
    if result is None:
        print("-"*60)
        return None
    u, v = result
    recon = wedge_product(u,v)
    ok = all(coef(recon,i,j) == coef(w,i,j) for i,j in combinations(range(4),2))
    print("Reconstructed:", recon)
    print("Original     :", w)
    print("Reconstruction OK" if ok else "MISMATCH -- check formula")
    print("-"*60)
    return ok

####################################################################
# Tests
####################################################################
if __name__ == "__main__":
    print("Test 1: non-decomposable (e0^e1 + e2^e3)")
    verify_decompose({(0,1):1, (2,3):1})

    print("Test 2: Case 1")
    verify_decompose({(0,3):1, (1,3):2, (2,3):-1})

    print("Test 3: Case 2 (a14!=0, a12!=0)")
    verify_decompose({(0,1):1, (0,2):2, (0,3):4, (1,2):1, (1,3):3, (2,3):2})
```

```
print("Test 4: Case 3 (a14!=0, a12=0, a13!=0)")
verify_decompose({(0,2):3, (0,3):2, (1,2):6, (1,3):4, (2,3):5})

print("Test 5: Case 5 (a14=0, a12!=0)")
verify_decompose({(0,1):2, (0,2):3, (1,2):5, (1,3):6, (2,3):9})

print("Test 6: Case 6 (a14=0, a12=0, a13!=0)")
verify_decompose({(0,2):4, (1,2):7, (2,3):5})

print("Test 7: Case 7 (a14=0, a12=0, a13=0, a23!=0)")
verify_decompose({(1,2):3, (1,3):6, (2,3):9})
```

Out[1]:
```
Test 1: non-decomposable (e0^e1 + e2^e3)
Plucker relation NOT satisfied -> omega is non-decomposable.
  value = [{'Value': 1}]
------------------------------------------------------------
Test 2: Case 1
Case 1: a12=a13=a23=0
omega = u ^ v
  u = (1, 2, -1, 0)
  v = (0, 0, 0, 1)
Reconstructed: {(0, 3): 1, (1, 3): 2, (2, 3): -1}
Original     : {(0, 3): 1, (1, 3): 2, (2, 3): -1}
Reconstruction OK
------------------------------------------------------------
Test 3: Case 2 (a14!=0, a12!=0)
Case 2: a14!=0, a12!=0
omega = u ^ v
  u = (1, 3/4, 1/2, 0)
  v = (0, 1, 2, 4)
Reconstructed: {(0, 1): 1, (0, 2): 2, (0, 3): 4, (1, 2): 1, (1, 3): 3, (2, 3): 2}
Original     : {(0, 1): 1, (0, 2): 2, (0, 3): 4, (1, 2): 1, (1, 3): 3, (2, 3): 2}
Reconstruction OK
------------------------------------------------------------
Test 4: Case 3 (a14!=0, a12=0, a13!=0)
Case 3: a14!=0, a12=0, a13!=0
omega = u ^ v
  u = (1, 2, 5/2, 0)
  v = (0, 0, 3, 2)
Reconstructed: {(0, 2): 3, (0, 3): 2, (1, 2): 6, (1, 3): 4, (2, 3): 5}
Original     : {(0, 2): 3, (0, 3): 2, (1, 2): 6, (1, 3): 4, (2, 3): 5}
Reconstruction OK
------------------------------------------------------------
Test 5: Case 5 (a14=0, a12!=0)
Case 5: a14=0, a12!=0
omega = u ^ v
  u = (0, 2, 3, 0)
  v = (-1, 0, 5/2, 3)
Reconstructed: {(0, 1): 2, (0, 2): 3, (1, 2): 5, (1, 3): 6, (2, 3): 9}
Original     : {(0, 1): 2, (0, 2): 3, (1, 2): 5, (1, 3): 6, (2, 3): 9}
Reconstruction OK
------------------------------------------------------------
Test 6: Case 6 (a14=0, a12=0, a13!=0)
Case 6: a14=0, a12=0, a13!=0
omega = u ^ v
  u = (0, 0, 4, 0)
  v = (-1, -7/4, 0, 5/4)
```

```
Reconstructed: {(0, 2): 4, (1, 2): 7, (2, 3): 5}
Original     : {(0, 2): 4, (1, 2): 7, (2, 3): 5}
Reconstruction OK
------------------------------------------------------------
Test 7: Case 7 (a14=0, a12=0, a13=0, a23!=0)
Case 7: a14=0, a12=0, a13=0, a23!=0  [corrected]
omega = u ^ v
  u = (0, 1, 0, -3)
  v = (0, 0, 3, 6)
Reconstructed: {(1, 2): 3, (1, 3): 6, (2, 3): 9}
Original     : {(1, 2): 3, (1, 3): 6, (2, 3): 9}
Reconstruction OK
------------------------------------------------------------
```